Divide and Conquer (and Binary Search)

For UQCS Competitive Programming Group

Slides by Fries



UQ Computing Society

Divide and Conquer

Divide and Conquer is a general algorithm design paradigm that consists of 3 main components: Divide, Conquer and Combine.

- Divide
  - Break down the problem into smaller subproblems.
- Conquer
  - Solve the subproblems independently (usually recursively).
- Combine/Merge
  - Combine the subproblem solutions into a solution for the original problem.

Some examples of Divide and Conquer include Binary Search, Merge Sort and Quick Sort.

Example: Merge Sort

- Merge-sort on an input sequence S with n elements consists of three steps:
  - Divide: partition S into two sequences S_1 and S_2 of about $n/2$ elements each
  - Conquer: recursively sort S_1 and S_2
  - Combine: merge S_1 and S_2 into a unique sorted sequence

```
Algorithm mergeSort(S, C)
  Input sequence S with n
        elements, comparator C
  Output sequence S sorted
        according to C
  if S.size() > 1
        (S1, S2) ← partition(S, n/2)
        mergeSort(S1, C)
        mergeSort(S2, C)
        S ← merge(S1, S2)'
```

Credit to Goodrich, Tamassia

Master Method (Extended reading) Complexity of Divide and Conquer Algorithms

- Most divide-and-conquer recurrence relation have the form:

$$T(n) = \begin{cases} c & if\ n < d \\ aT\left(\frac{n}{b}\right) + f(n) & if\ n \geq d \end{cases}$$

- The master theorem states that:
  - If $f(n)$ is $O(n^{\log_b a - \varepsilon})$, then $T(n)$ is $\Theta(n^{\log_b a})$
  - If $f(n)$ is $\Theta\left(n^{\log_b a} \log^k n\right)$, then $T(n)$ is $\Theta\left(n^{\log_b a} \log^{k+1} n\right)$
  - If $f(n)$ is $\Omega(n^{\log_b a + \varepsilon})$, then $T(n)$ is $\Theta\left(f(n)\right)$, provided $af\left(\frac{n}{b}\right) \leq \delta f(n)$, for some $\delta < 1$.

Binary Search

- Canonically, Binary Search is used to search for an item in a *sorted array*. We check the middle element of the array to determine if it is the item we're looking for.
- We then divide our array into left and right (divide) and search one side (conquer). The complexity of the algorithm is $O(logn)$.

- We'll look at a bit more 'uncanonical' approach to binary search problem

Bisection Method

Problem:

- You buy a car with loan and want to pay the loan in monthly instalments of ***d*** dollars for ***m*** months.
- Suppose the value of the car is originally ***v***, the bank charges an interest rate of ***i%*** for any unpaid plan at the end of each month.
- What is the amount of money ***d*** that you must pay per month (to 2 digit after the decimal point)?

Bisection Method

a	**b**	**d** $= \frac{a+b}{2}$	**status:** $f(d, m, v, i)$	**action**
0.01	1100.00	550.005	undershoot by 54.9895	increase d
550.005	1100.00	825.0025	overshoot by 522.50525	decrease d
550.005	825.0025	687.50375	overshoot by 233.757875	decrease d
550.005	687.50375	618.754375	overshoot by 89.384187	decrease d
550.005	618.754375	584.379688	overshoot by 17.197344	decrease d
550.005	584.379688	567.192344	undershoot by 18.896078	increase d
567.192344	584.379688	575.786016	undershoot by 0.849366	increase d
...	...	...	a **few** iterations later ...	...
...	...	576.190476	stop; error is now less than ϵ	answer = 576.19

Table 3.1: Running Bisection Method on the Example Function

- Solvable by binary search!
- Pick a reasonable range as a starting point.
- Example: m = 2, v = 1000, i = 10%.
- Take a = 0.01 and b = (1 + 0.1) * 1000 as starting point.

Competition Time!

- Todays link (already posted in discord hopefully):

www.hackerrank.com/cpg-17092025

- (Also I literally format them the same way each week)

References

- Competitive Programming 3 - Felix Halim and Steven Halim
- Algorithm Design and Applications, by M. T. Goodrich and R. Tammasia, Wiley